# PADCOM V22 - GUIA DE SEGURANçA E PERFORMANCE

**Versão:** 1.0

**Data:** 11 de Abril de 2026

---

## SEGURANÇA

### 1. Validação de Entrada

Todas as rotas usam Zod para validação de entrada:

```TypeScript
import { validateBody, validateQuery } from "@workspace/validations_v22";

router.post(
  "/v22/checkin/patologias",
  validateBody(CheckinPatologiasRequestSchema),
  checkinPatologiasHandler
);
```

**Validações implementadas:**

- IDs devem ser números positivos
- Intensidades devem estar entre 0-5
- Datas devem ser válidas (ISO 8601)
- Strings têm limite de caracteres
- Arrays têm limites de tamanho

### 2. Autenticação e Autorização

**Implementar:**

```TypeScript
import { auth } from "@workspace/auth";

router.post(
  "/v22/checkin/patologias",
  auth.middleware, // Verificar JWT
  auth.authorize(["PACIENTE", "MEDICO", "ADMIN"]),
```

```
  validateBody(CheckinPatologiasRequestSchema),
  checkinPatologiasHandler
);
```

**Regras de acesso:**

- PACIENTE: Pode fazer check-in próprio, ver seu dashboard
- ENFERMEIRA: Pode responder cards, ver timeline
- MEDICO: Acesso completo ao dashboard
- ADMIN: Acesso total ao sistema

## 3. Rate Limiting

**Implementar com express-rate-limit:**

TypeScript

```typescript
import rateLimit from "express-rate-limit";

const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutos
  max: 100, // 100 requisições por IP
  message: "Muitas requisições, tente novamente mais tarde",
});

app.use("/api/v22/", limiter);

// Limites específicos para operações sensíveis
const strictLimiter = rateLimit({
  windowMs: 60 * 1000, // 1 minuto
  max: 10, // 10 requisições por IP
});

router.post("/v22/checkin/patologias", strictLimiter, ...);
```

## 4. SQL Injection Prevention

Usar Drizzle ORM (já previne SQL injection):

TypeScript

```typescript
// ✅ Seguro - Drizzle ORM
const pacientes = await db
  .select()
  .from(pacientesTable)
```

```
  .where(eq(pacientesTable.id, pacienteId));

// ❌ Nunca fazer isso
const query = `SELECT * FROM pacientes WHERE id = ${pacienteId}`;
```

## 5. CORS

**Configurar CORS corretamente:**

TypeScript

```
import cors from "cors";

app.use(
  cors({
    origin: [
      "https://padcom.clinica.padua.com",
      "https://app.padcom.clinica.padua.com",
    ],
    credentials: true,
    methods: ["GET", "POST", "PUT", "DELETE"],
    allowedHeaders: ["Content-Type", "Authorization"],
  } )
);
```

## 6. HTTPS

**Em produção, sempre usar HTTPS:**

TypeScript

```
app.use((req, res, next) => {
  if (process.env.NODE_ENV === "production" && !req.secure) {
    return res.redirect(`https://${req.headers.host}${req.url}` );
  }
  next();
});
```

## 7. Dados Sensíveis

**Nunca logar dados sensíveis:**

TypeScript

```
// ❌ Errado
console.log("Paciente:", paciente); // Pode conter dados sensíveis

// ✅ Correto
console.log("Paciente criado:", { id: paciente.id, nome: paciente.nome });
```

## 8. Variáveis de Ambiente

### Usar .env para dados sensíveis:

Bash

```
# .env
DATABASE_URL=postgresql://user:password@localhost:5432/padcom
JWT_SECRET=seu_secret_muito_seguro_aqui
API_KEY=sua_api_key_aqui
NODE_ENV=production
```

### Nunca commitar .env:

Bash

```
# .gitignore
.env
.env.local
.env.*.local
```

---

# PERFORMANCE

## 1. Índices do Banco de Dados

### Índices já criados nas migrations:

SQL

```
-- Índices para queries frequentes
CREATE INDEX idx_paciente_patologias_checkin_paciente_id
  ON paciente_patologias_checkin(paciente_id);

CREATE INDEX idx_paciente_patologias_checkin_ativo
  ON paciente_patologias_checkin(ativo);

CREATE INDEX idx_acompanhamento_timeline_paciente_data
```

```
  ON acompanhamento_timeline(paciente_id, data_evento DESC);

CREATE INDEX idx_acompanhamento_cards_paciente_status
  ON acompanhamento_cards_mensais(paciente_id, status);
```

## 2. Query Optimization

** Usar select() específico, não SELECT :*

TypeScript

```
// ✅ Correto - Seleciona apenas campos necessários
const patologias = await db
  .select({
    id: patologiasMasterTable.id,
    nome: patologiasMasterTable.nome,
    intensidade: pacientePatologiasCheckinTable.intensidade0_5,
  })
  .from(pacientePatologiasCheckinTable)
  .leftJoin(
    patologiasMasterTable,
    eq(pacientePatologiasCheckinTable.patologiaId, patologiasMasterTable.id)
  )
  .where(eq(pacientePatologiasCheckinTable.pacienteId, pacienteId));

// ❌ Errado - Seleciona tudo
const patologias = await db
  .select()
  .from(pacientePatologiasCheckinTable)
  .leftJoin(patologiasMasterTable, ...);
```

## 3. Paginação

**Implementar paginação para grandes datasets:**

TypeScript

```
const ITEMS_PER_PAGE = 20;

router.get("/v22/acompanhamento/timeline/:pacienteId", async (req, res) => {
  const { pagina = 1, limite = ITEMS_PER_PAGE } = req.query;
  const offset = (pagina - 1) * limite;

  const timeline = await db
    .select()
    .from(acompanhamentoTimelineTable)
```

```
    .where(eq(acompanhamentoTimelineTable.pacienteId, pacienteId))
    .orderBy(desc(acompanhamentoTimelineTable.dataEvento))
    .limit(limite)
    .offset(offset);

  const total = await db
    .select({ count: sql`count(*)` })
    .from(acompanhamentoTimelineTable)
    .where(eq(acompanhamentoTimelineTable.pacienteId, pacienteId));

  res.json({
    timeline,
    paginacao: {
      pagina,
      limite,
      total: total[0].count,
      totalPaginas: Math.ceil(total[0].count / limite),
    },
  });
});
```

## 4. Caching

**Implementar Redis para dados que não mudam frequentemente:**

TypeScript

```
import redis from "redis";

const client = redis.createClient();

router.get("/v22/patologias-master", async (req, res) => {
  // Tentar buscar do cache
  const cached = await client.get("patologias-master");
  if (cached) {
    return res.json(JSON.parse(cached));
  }

  // Se não estiver em cache, buscar do banco
  const patologias = await db
    .select()
    .from(patologiasMasterTable)
    .where(eq(patologiasMasterTable.ativo, true));

  // Armazenar em cache por 1 hora
  await client.setEx("patologias-master", 3600, JSON.stringify(patologias));
```

```
  res.json({ total: patologias.length, patologias });
});
```

## 5. Connection Pooling

**Configurar pool de conexões PostgreSQL:**

TypeScript

```
import { Pool } from "pg";

const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  max: 20, // Máximo de conexões
  idleTimeoutMillis: 30000,
  connectionTimeoutMillis: 2000,
});
```

## 6. Monitoramento de Performance

**Adicionar logging de performance:**

TypeScript

```
function logPerformance(operacao: string, duracao: number) {
  if (duracao > 1000) {
    console.warn(`⚠️  ${operacao} levou ${duracao}ms`);
  } else {
    console.log(`✅ ${operacao} completado em ${duracao}ms`);
  }
}

router.get("/v22/dashboard/paciente/:pacienteId", async (req, res) => {
  const inicio = Date.now();

  // ... código da rota ...

  const duracao = Date.now() - inicio;
  logPerformance("Dashboard do paciente", duracao);

  res.json(dashboard);
});
```

## 7. Compressão de Resposta

**Usar gzip para comprimir respostas:**

```typescript
import compression from "compression";

app.use(compression());
```

## 8. Lazy Loading

**Carregar dados relacionados sob demanda:**

```typescript
// ❌ Errado - Carrega tudo
const paciente = await db
  .select()
  .from(pacientesTable)
  .leftJoin(patologiasTable, ...)
  .leftJoin(sintomasTable, ...)
  .leftJoin(cirurgiasTable, ...)
  .leftJoin(cardsTable, ...);

// ✅ Correto - Carrega sob demanda
const paciente = await db
  .select()
  .from(pacientesTable)
  .where(eq(pacientesTable.id, pacienteId));

// Depois, se necessário:
const patologias = await db
  .select()
  .from(pacientePatologiasCheckinTable)
  .where(eq(pacientePatologiasCheckinTable.pacienteId, pacienteId));
```

---

# BENCHMARKS ESPERADOS

| Operação | Tempo Esperado | Limite Máximo |
|---|---|---|
| GET /patologias-master | < 100ms | 500ms |
| POST /checkin/patologias | < 200ms | 1000ms |
| GET /dashboard/paciente/:id | < 500ms | 2000ms |

| | | |
|---|---|---|
| GET /acompanhamento/timeline/:id | < 300ms | 1500ms |
| POST /motor-clinico/processar-checkin | < 400ms | 2000ms |

---

## CHECKLIST DE SEGURANÇA

- [ ] Todas as rotas têm validação Zod
- [ ] Autenticação JWT implementada
- [ ] Rate limiting configurado
- [ ] CORS configurado corretamente
- [ ] HTTPS ativado em produção
- [ ] Dados sensíveis em variáveis de ambiente
- [ ] .env não está commitado
- [ ] Logs não contêm dados sensíveis
- [ ] SQL injection prevenido (usando ORM)
- [ ] Testes de segurança executados

---

## CHECKLIST DE PERFORMANCE

- [ ] Índices do banco criados
- [ ] Queries otimizadas (select específico)
- [ ] Paginação implementada
- [ ] Caching Redis configurado
- [ ] Connection pooling ativo
- [ ] Compressão gzip ativa
- [ ] Lazy loading implementado
- [ ] Monitoramento de performance ativo
- [ ] Benchmarks validados

- [ ] Load testing realizado

---

**Implementar esses pontos garante um sistema seguro, rápido e escalável.**